2 Feature Mappings for Image Reconstruction [30 Marks]

This question is a continuation of the previous one where we attempted to reconstruct an image directly from raw inputs. Recent work Tancik et al., 2020 demonstrated that mapping inputs through **Fourier feature expansions** enables neural networks to approximate high-frequency functions much more effectively. In contrast, using raw inputs or low-order polynomial (Taylor) expansions provides different inductive biases and captures different aspects of the data.

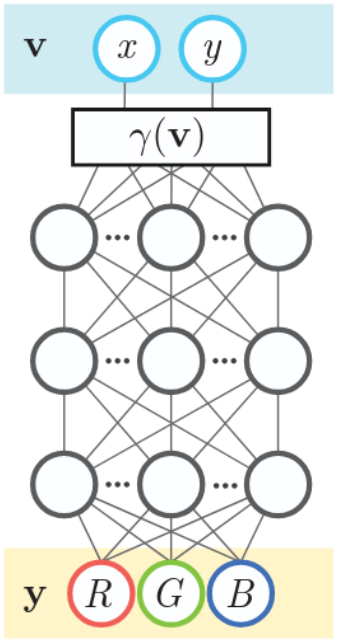


Figure 2: Illustration of an MLP with Fourier feature mapping applied to 2D input coordinates (x, y).

We will compare three different feature representations when training an MLP. The idea is to apply a transformation $\gamma(\mathbf{v})$ on the input pixel/coordinate vector $\mathbf{v} = (x, y)$ before feeding it into the network:

1. **Raw Pixels**: directly use pixel coordinates or intensities.
2. **Taylor Series Expansion**: approximate coordinates using polynomial basis functions.
3. **Fourier Feature Expansion**: map coordinates into a sinusoidal embedding.

2.1 Setup and Visualization [4]

- Load a grayscale image (`smiley.png`) from the following link, which is already 256×256: smiley.png

- Load an RGB image (`cat.jpg`) from the link below and resize it to 256×256: cat.jpg
- The task is to reconstruct these images using the three feature mappings: Raw, Polynomial, and Fourier.

2.2 Feature Expansions [9]

Raw Mapping The simplest approach is to directly use the 2D coordinates or pixel values as input features to the network. This mapping does not introduce any additional basis functions or transformations:

$$\gamma_{\text{raw}}(x, y) = (x, y)$$

```
def get_raw(coords):
    return coords
```

Polynomial Expansion (Taylor-inspired) Instead of using the full Taylor series, we construct a simple polynomial basis up to order k to approximate nonlinear variations in the input coordinates. This expansion allows the network to represent higher-order relationships between x and y without explicitly computing derivatives:

$$\gamma_{\text{poly}}(x, y) = \left[x, y, x^2, y^2, xy, \ldots, x^k, y^k\right]$$

```
def get_polynomial(coords, order=5):
    # Expands into [x, y, x^2, y^2, xy, ..., x^order, y^order]
```

This polynomial expansion is used to approximate a Taylor series and gives the network richer input features compared to raw coordinates.

Fourier Features Including all cross terms in a true 2D Fourier expansion i.e., all combinations of sine and cosine functions of x and y would drastically increase computation and memory usage. To avoid this combinatorial explosion, we use a practical trick: we compute sinusoidal embeddings for each coordinate independently and then stack them together. Concretely, we take

$$\gamma_{\text{fourier}}(x, y) = \begin{bmatrix} 1, \sin(2\pi f_1 x), \cos(2\pi f_1 x), \ldots, \sin(2\pi f_k x), \cos(2\pi f_k x), \\ \sin(2\pi f_1 y), \cos(2\pi f_1 y), \ldots, \sin(2\pi f_k y), \cos(2\pi f_k y) \end{bmatrix}$$

```
def get_fourier(coords, freq=10):
    # Practical Fourier feature approximation:
    # stack 1, sin/cos of x, sin/cos of y
    # Avoids cross-term combinatorial explosion
```

This approach keeps the number of features manageable while still capturing high-frequency variations along both axes.

2.3 Normalization and Modular MLP [3]

Each feature expansion produces values on very different scales, which can strongly affect training. To make fair comparisons across Raw, Polynomial, and Fourier mappings, proper normalization is essential.

- **Raw features:** scale pixel coordinates or intensities to $[0, 1]$.
- **Polynomial features:** higher-order terms (e.g., x^k, y^k) can grow very large; rescale these to avoid exploding values.
- **Fourier features:** sine and cosine outputs lie in $[-1, 1]$. Input coordinates should be scaled accordingly to roughly cover one full cycle of the sinusoidal functions - consider how this choice affects aliasing and high-frequency representation.

Define a modular data loader:

```
Modular_Dataloader(img_path, image_type, method, order, freq)
```

It accepts the following arguments:

- `img_path`: path to the image (RGB or grayscale)
- `image_type`: either `"RGB"` or `"Gray"`
- `method`: feature mapping to use (`"Raw"`, `"Polynomial"`, or `"Fourier"`)
- `order`: order of polynomial expansion (if using Polynomial)
- `freq`: number of Fourier frequencies (if using Fourier)

The loader produces the corresponding input features ready for the MLP. You can use the same MLP from the previous question and modify it slightly to accept this dataloader as input.

2.4 Training and Comparison [4]

We now use all the building blocks from previous sections to train and compare the different feature mappings. The training setup is as follows:

- Use the same MLP architecture for all methods. For example, a 3-layer MLP with hidden sizes $[64, 128, 128]$. Keep the architecture constant to allow a fair comparison.
- Use the same number of epochs for each image:
  - 50 epochs for `smiley.png`
  - 150 epochs for `cat.jpg`
- Use the same loss function as in the previous question (e.g., MSE between predicted and target pixel values).

Baseline: Raw Features : Train the MLP using raw pixel coordinates and save the predicted output at each epoch.

Polynomial Features : Train with different polynomial orders: $[5, 15, 25]$ and record the images for each of them .

Fourier Features : Similarly, train using Fourier feature expansions with different numbers of frequencies: $[5, 15, 25]$. Record the images

Tabulation of Results Create a table comparing:

- Method (Raw / Polynomial / Fourier)
- Final Inference Loss
- Epoch time
- Number of input parameters

Visualization with GIF : To better understand convergence and efficiency:

- Pick one working combination each of Polynomial and Fourier along with Raw features for the same architecture and number of epochs.
- Use the saved images for each epoch to create a 1×3 subplot GIF:
  - Left: Raw
  - Middle: Polynomial
  - Right: Fourier
- Include a legend in each subplot showing the loss curve.
- You can generate the GIF using the `PIL` library or any other preferred tool.

This visualization allows us to directly compare how each feature mapping affects convergence speed and reconstruction quality over training.

2.5 Reconstruction on Blurred Images [10]

For this exercise, we consider a dataset consisting of the same cat image blurred with Gaussian blur levels $\sigma = 0, 1, \ldots, 10$. The images are numbered from `blur_0.jpg` to `blur_10.jpg` and can be accessed here.

Train the coordinate MLP to reconstruct all blurred images using the following methods:

- **BASE:** Using raw input coordinates
- **FOURIER:** Using Fourier feature expansion with frequency $k = 5$

Training Instructions

1. Use early stopping as implemented previously to determine convergence. You may start with a maximum of 100 epochs for each image.

2. For each image and method, record:

    - The reconstructed image
    - The final reconstruction loss

Analysis and Visualization

1. Plot the final reconstruction loss across all images:

    - X-axis: image index or blur level (`blur_0` to `blur_10`)
    - Y-axis: reconstruction loss
    - Include both BASE and FOURIER results on the same plot

2. Plot the same comparison using a logarithmic scale for the Y-axis to highlight differences in low-loss regimes.

Discussion

- Observe how the reconstruction performance changes with increasing blur. Explain the observed behavior in terms of how high-frequency details are lost as blur increases, and verify your reasoning by inspecting the predicted images.